Content Pipeline

Content Pipeline

22 INTERVIEW PROBES · DEEP REHEARSAL Q&A

MEMORY MODEL UNDER STREAMING

Q1. Export a 1,000,000-row CSV without OOM. How?

Server-side cursor (`pg-query-stream` , batch 100) piped to the response. Backpressure throttles the cursor to the client's drain rate, so memory is **constant** regardless of row count.

> **Follow-up:** What sets the batch size — why 100, not 1 or 100,000?
> It's fetch granularity, not the memory ceiling. **batch=1** is a round-trip per row (crawls); **batch=100,000** holds a huge fetch buffer in RAM and stalls in bursts. 100 amortizes round-trips while staying tiny.

> **Follow-up:** The client is on slow 3G and stops reading mid-download. What is your server doing right now?
> TCP window fills → `res.write()` returns false → transform stops → cursor stops → Postgres holds the cursor **open and idle**, pinning a connection. A slow-reader can exhaust the pool (slowloris). Fix: statement timeout, or stream to S3 + presigned URL.

What sounds senior: Saying memory is constant **because of backpressure** — not just "I'd stream it" — is the senior tell. Bonus for naming the held-cursor connection risk before being asked.

CROSS-STORE CONSISTENCY

Q2. Import writes to Postgres AND S3. The DB write fails. What happens?

Dual-write hole: the DB transaction rolls back, but the S3 objects don't — they're orphaned. Fix: track every created key, delete them in the `catch` .

> **Follow-up:** Your compensating delete *also* fails — S3 is throttling. Now what?
> Don't block the user request on best-effort cleanup. Record orphan keys to a durable cleanup queue and retry async with backoff. The real backstop is a **reconciler** sweeping S3 keys with no DB row.

> **Follow-up:** Make the whole import idempotent so a retry is always safe.
> Content-hash keys (same bytes = same key → re-upload is a no-op), a DB **upsert** on a deterministic key, and the ladder's `name+version` REMAP to block duplicate logical rows. A retried import **converges**.

> **Follow-up:** Is there a way to make the two writes atomic instead of reconciling after the fact?
> Yes — the **transactional outbox**, and reaching for it is the senior move because it kills the race instead of healing it. The DB write and an **outbox-event** row commit in **one transaction** — either both land or neither does, so there's never a window where Postgres and S3 disagree. A separate **relay** reads the outbox and does the S3 work with **retries**, marking each event done. You've collapsed a two-store dual-write into a **single atomic commit point** plus an idempotent relay. The catch: it needs a transactional store to anchor on — when the two stores genuinely can't share a transaction, the reconciler is the honest backstop. So I'd frame it as **outbox to prevent, reconciler to heal**, and name which one the constraints allow.

What sounds senior: Volunteering the **reconciler as the real backstop** — not just the catch-block delete — shows you've operated this in production, not only designed it on a whiteboard.

I/O & CONCURRENCY REASONING

Q3. Why hash the file *while* uploading instead of after it's stored?

A `PassThrough` forks **one** read into hash + upload. Hashing after = 2× disk reads; buffering-then-both = the whole file in RAM. The fork is the only option that survives a 500 MB package.

> **Follow-up:** Some S3 APIs want Content-MD5 / length up front — but you don't know the hash until the stream ends. Tension?
>
> That's why **multipart** exists — no total length or whole-object hash needed up front; parts stream and S3 assembles. If an API truly needs the digest first, you must buffer or two-pass — the precise trade you're avoiding.

> **Follow-up:** Two uploads of identical bytes race and compute the same key. What happens?
>
> On S3, last-write-wins on the same key — harmless, since content is identical. The contention point is the **DB dedup row**: `ON CONFLICT` on the hash so both converge to one row.

What sounds senior: Surfacing the **Content-MD5 tension yourself**, before being asked, signals you know exactly where the streaming abstraction leaks.

ARCHITECTURE TRADE-OFFS & SCALING CEILINGS

Q4. Why a Lambda per object instead of SQS + a worker pool?

Zero ops and low latency at this volume. **Switch condition:** move to SQS the moment you need retries, a DLQ, or ordering. Naming the switch condition is what scores.

> **Follow-up:** S3 delivers the event twice (at-least-once). Walk me through preventing a double-processed file.
>
> Idempotent processing: a content-hash key + upsert, or a **processed-marker** (a DynamoDB conditional put on the key) the handler checks first. Never rely on exactly-once delivery.

> **Follow-up:** 10,000 objects land in one second. What breaks first, and what's your ceiling?
>
> **Lambda concurrency** throttles first → events retry / DLQ. Then the **DB pool** and **S3 PUT rate per prefix** saturate. Ceilings: concurrency (RDS Proxy / queue), connections, per-prefix rate (spread keys). At firehose scale, buffer through SQS/Kinesis.

What sounds senior: Giving the **switch condition** — the exact threshold where you'd move to SQS — instead of defending one choice is the senior move. Naming concrete ceilings seals it.

DATA MODELING & IDEMPOTENT IMPORT

Q5. Why ship the export as a SQLite bundle instead of JSON?

SQLite keeps **real foreign keys** and stores BLOBs at **1:1** size. JSON base64-bloats binary ~33% and loses referential integrity — you'd rebuild the graph by hand on import.

> **Follow-up:** On import, ids collide with existing rows. Your algorithm?
>
> The 4-tier ladder: `id+company` REUSE, `name+version` REMAP, `id-taken` REGEN, else INSERT — record `oldId→newId` and rewrite child FKs before insert.

> **Follow-up:** Why check name+version BEFORE the id-collision rule?
>
> So a re-import **de-dupes** (REMAP to existing) instead of forking a new uuid (REGEN). Swap the order and every re-import silently **doubles** your data.

What sounds senior: Explaining **why** name+version is checked first — the de-dupe-vs-fork consequence — is what separates "I wrote an importer" from "I designed one."

IRREVERSIBLE-DECISION FORESIGHT

Q6. You set the S3 trigger prefix to `uploads/`. What's the future risk?

S3 notifications **forbid overlapping prefixes**, so you can't later split `uploads/` into `uploads/images/` without reconfiguring everything. One-way door — design the taxonomy before the first trigger ships.

> **Follow-up:** You're forced to add per-type routing later anyway. How, without re-architecting?
>
> Move routing out of **S3 config** (rigid) into **code**: one trigger on `uploads/`, dispatch inside the Lambda by key sub-prefix or metadata — or fan out (Lambda → an SQS queue per type). Flexibility lives in code, not the bucket.

What sounds senior: Calling it a **one-way door** and pre-deciding the taxonomy shows you reason about **irreversibility** — a staff-level instinct most candidates skip.

DESIGN ADAPTABILITY

Q7. Product wants live video *transcoding* added (not just thumbnails). How does your pipeline change?

Transcoding is long-running and CPU/GPU-heavy — it doesn't belong on Lambda (15-min cap, no GPU). Keep the **strategy-dispatch pattern**: add a video handler, but instead of processing inline it **enqueues a job** to ECS / MediaConvert. The S3 event still triggers; the handler fans out and the status flow gains a PROCESSING → TRANSCODED state.

> **Follow-up:** Now a live-preview feature needs that content processed in under 2 seconds. What changes?
>
> The async S3-event path is eventual — too slow. Add a **synchronous fast-path**: small previews process inline in the request and skip the queue; heavy/bulk stays async. Two-tier by interactivity. Cache by content hash so repeat previews are instant.

> **Follow-up:** How do you keep ONE codebase serving both the fast sync path and the heavy async path?
>
> The dispatch map stays the single entry point; the handler branches on a **size/mode flag** — inline for small+interactive, enqueue for large+bulk. Same contract, same handlers; only the execution venue differs. The strategy pattern earning its keep.

What sounds senior: Extending the design **along its grain** — preserving the dispatch pattern while moving heavy work to the right compute, rather than bolting on a parallel system — plus naming the sync/async two-tier split, is the senior instinct.

SECURITY BOUNDARY & LEAST-PRIVILEGE

Q8. A client's upload credential leaks. What can an attacker do, and how do you shrink the blast radius?

A broad credential reads the whole bucket and overwrites anything. Shrink it with a **presigned URL** scoped to one key + a short TTL (minutes) — a leaked URL is then one upload, not the bucket. The processing role is separate and read-only on the upload prefix.

> **Follow-up:** A presigned URL still lets anyone holding it upload until it expires. Tighten it further.
>
> Bake conditions into the policy: `Content-Length-Range` caps size, `Content-Type` pins the format, a per-user key prefix isolates tenants. The URL becomes a **single-purpose token** — one file, one size, one type, one place.

> **Follow-up:** A malicious upload (zip bomb, polyglot) reaches a processor. Stop it taking down the fleet.
>
> Validate before processing, not at upload: cap decompressed size, check magic bytes against the claimed type, and run processing in an **isolated, resource-capped sandbox** (separate function/container with memory + time limits) so one bad object can't take the fleet down.

What sounds senior: Turning a credential into a **single-purpose token** with size/type/prefix conditions — and sandboxing untrusted input — is the containment instinct senior interviewers probe for.

OBSERVABILITY & OPERABILITY

Q9. It's 3 AM, the pipeline is 'slow,' and you have no logs in front of you. What four signals do you wish you'd instrumented?

Queue depth / event age (is work piling up?), **processing latency p99** (are jobs individually slow?), **error + DLQ rate** (are things failing?), and **reconciler orphan count** (is the dual-write leaking?). Those four localize almost any incident to a stage.

> **Follow-up:** Queue depth is climbing but error rate is flat. What does that tell you?
>
> Not failures — **throughput**. Consumers can't keep up with arrival rate: either a downstream dependency slowed (raising per-job latency) or concurrency is throttled. You scale consumers or fix the slow dependency — chasing errors finds nothing.

> **Follow-up:** What's the one thing you page a human for, versus just log?
>
> Page on **sustained DLQ growth** or **event age past SLA** — those mean work is silently not getting done. Transient spikes that self-heal via retry are log-only; paging on them is alert fatigue. Alert on *symptoms users feel*, not every internal hiccup.

What sounds senior: Choosing the **four signals that localize an incident** and splitting page-worthy (SLA breach, DLQ growth) from log-only (self-healing retries) shows you've carried a pager, not just shipped features.

THE EXACTLY-ONCE ILLUSION

Q10. A processor pulls a message, does the work, then crashes *before* deleting it. It redelivers. How do you avoid doing the work twice?

You can't buy exactly-once **delivery** — you engineer idempotent **effects**. Make the write a no-op on replay: a **processed-marker** (conditional put on the content/message key) the handler checks-and-sets first, so the second delivery sees 'already done' and acks without re-running.

> **Follow-up:** The side effect can't be made idempotent — it charges a card / sends an email. Now what?
>
> Put it behind an **idempotency key** the downstream honors (Stripe-style): same key, same charge, executed once. If the downstream can't dedupe, use a **transactional outbox** — record intent atomically with the state change, then a relay delivers with dedup downstream.

> **Follow-up:** Where do you store the marker so the check and the work don't themselves race?
>
> They must commit **atomically** or the check is a TOCTOU bug. Either the marker lives in the same DB transaction as the write (one commit), or the conditional put *is* the lock — first writer wins the key, losers back off. The marker can never be a separate best-effort call.

> **Follow-up:** The marker table grows forever — one row per message ever processed. You want to expire old markers, but won't that let a late redelivery slip through and double-process?
>
> Yes — and that's the real tradeoff, not a detail. I TTL the markers to a window **longer than the maximum redelivery horizon** — the queue's retention plus its visibility-timeout ceiling — so anything that could still redeliver still finds its marker. Past that horizon a redelivery is effectively impossible, so expiring is safe. For effects that are **catastrophic to repeat** — a charge — I don't TTL at all: keep the key permanently, and if storage is the worry, age cold keys into a cheaper tier or a bloom filter for the 'definitely seen' check. The point: marker retention is a **correctness** parameter, not a cleanup job — you size it against the redelivery window, never shorter.

What sounds senior: Naming that exactly-once **delivery** is impossible and you build idempotent **effects** instead — plus the atomic check-and-set so the marker doesn't itself race — is the distributed-systems maturity Staff rounds dig for.

LARGE-OBJECT TRANSFER MECHANICS

Q11. A 500 MB upload fails at 80% over flaky mobile. Does the user restart from zero?

Not with **multipart upload**: the file is split into parts (e.g. 8 MB), each uploaded and ack'd independently, so a failed part retries alone. The client resumes by re-sending only the missing parts; S3 assembles on `CompleteMultipartUpload`.

> **Follow-up:** The upload is abandoned at 80% and never completed. What's in your bucket, and what does it cost?
>
> Uncompleted multipart uploads leave **orphaned parts** — they consume storage but don't show in normal listings, so it's a silent bill. Fix: an **S3 lifecycle rule** to abort incomplete multipart uploads after N days. A classic cost leak.

> **Follow-up:** Does your S3 trigger fire per part or once — and why does it matter?
>
> Once — `s3:ObjectCreated:CompleteMultipartUpload` fires on assembly, not per part, so the handler sees one complete object, not 60 fragments. Subscribe to the complete event or you'll process partial garbage.

What sounds senior: Knowing multipart resumes **per part**, that abandoned uploads silently bill via orphaned parts (lifecycle rule to abort), and that the trigger fires once on completion — these specifics separate 'used S3' from 'operated S3 at scale.'

RECONCILER CORRECTNESS UNDER CONCURRENCY

Q12. Your reconciler deletes S3 keys with no DB row. An upload is *in flight* — object written, row not yet committed. Walk me through the bug and the fix.

Race: the reconciler sees an object with no row and deletes a perfectly good in-flight upload. First fix is a **grace period** — only reconcile objects older than, say, an hour (longer than any legitimate write window), so in-flight uploads are never eligible.

> **Follow-up:** A grace period is a guess. Make the reconciler correct, not just probably-correct.
>
> Make intent explicit: write a **pending marker** (a PENDING row, or an object tag) *before* the upload, flip to COMMITTED after. The reconciler deletes only objects with **no marker at all** — never a PENDING one. Correctness stops depending on a timing guess.

> **Follow-up:** The reconciler itself races a writer: reads 'no row,' writer commits, reconciler deletes. Still broken?
>
> Yes — a TOCTOU window. Close it with a **conditional delete** that re-checks for the row inside the delete, or have the writer claim the key first (PENDING) so the reconciler acts on an unambiguous marker, not a snapshot it may have read stale.

> **Follow-up:** To spread load, your reconciler checks 'is there a row?' against a **read replica**. What goes wrong?
>
> It deletes live data. A replica lags the primary, so 'no row' can mean 'the row committed but hasn't replicated yet' — the reconciler reads stale, sees an orphan that isn't one, and deletes an object whose record exists on the primary. The fix: the *delete decision* must read from the **primary**, or a read guaranteed to reflect committed writes — never a lagging replica. It can list candidate keys off a replica for cheapness, but it has to confirm against authoritative state before deleting. The rule worth saying out loud: a **destructive action can never be driven by an eventually-consistent read**.

What sounds senior: Spotting that the naive reconciler **deletes in-flight uploads**, rejecting the grace-period guess for an explicit **PENDING marker**, then closing the reconciler's own TOCTOU window — that layered correctness reasoning is the Staff-level depth this whole pipeline is built to probe.

COST MODELING & THE HIDDEN BILL

Q13. This pipeline processes 10M files a day. Where does the AWS bill actually go — and what's the line item that surprises people?

Not compute — **S3 requests and data transfer**. At 10M/day, PUTs alone run ~$50/day ($0.005 per 1K); GETs on every read, plus cross-AZ transfer when compute and bucket disagree on region, dwarf the Lambda-seconds. The surprise line is **NAT Gateway data processing** — a VPC Lambda reaching S3 without a gateway endpoint pays per-GB on top.

> **Follow-up:** Halve the cost without dropping a file. First move?
>
> An **S3 gateway VPC endpoint** — it's free, and it removes NAT and transfer charges for bucket traffic entirely. Then batch small files so you issue fewer PUTs, and add lifecycle rules to tier cold objects to S3-IA / Glacier. Compute is rarely the lever.

> **Follow-up:** Lambda vs Fargate for this — when does the cost curve flip?
>
> Lambda wins for **spiky, short, sub-second** work — you pay nothing idle. It flips to Fargate / ECS when invocations run long or go **sustained and high-volume**: past roughly one continuous vCPU of steady load, always-on Fargate beats per-ms Lambda. Duration × frequency is the deciding product.

What sounds senior: Naming **requests and transfer** — not compute — as the real bill, and reaching for a **gateway endpoint** before micro-optimizing code, shows you've read an actual AWS invoice. Knowing the Lambda→Fargate flip point seals it.

FORMAT & SCHEMA EVOLUTION

Q14. Your export is a CSV other teams' importers consume. You need to add a column. How do you ship it without breaking them?

Append-only, never reorder. Add the new column at the **end**, optional with a sane default — positional importers keep working, header-based ones pick it up. Breakage comes from **inserting or renaming** columns, which shifts every downstream index.

> **Follow-up:** An old importer must keep running for a year, but new consumers need a nested structure CSV can't express. Now what?
>
> **Version the format — don't mutate one.** Emit a `v2` (JSON / Parquet) alongside the frozen `v1` CSV; a **version field or path** selects the reader. Expand-then-contract: dual-publish, migrate consumers, retire v1 once its usage hits zero.

> **Follow-up:** How do you know when it's actually safe to retire v1?
>
> You don't guess — you **instrument it**. Emit a metric per format version on read, or track last-access per consumer. Retire v1 only after its read count has been zero for a full cycle. Schema retirement is a data-driven call, not a calendar one.

What sounds senior: Separating a **backward-compatible append** from a breaking reorder, escalating to **versioned formats with expand / contract** instead of mutating in place, and gating retirement on **measured usage** — that's how schema changes ship safely in production.

BACKPRESSURE THROUGH A FORKED STREAM

Q15. In the handler, one read forks to hash and S3 upload. Hashing is instant; the upload is slow. What happens to the pipe — and to memory?

The fork runs at the speed of its **slowest consumer**. The `PassThrough` feeding S3 fills to its `highWaterMark`, stops draining, and that backpressure propagates **back through the tee** to the source read — so the fast hash branch is throttled to the upload's pace. Memory stays bounded at the buffer size; unread chunks do **not** pile up.

> **Follow-up:** Someone 'fixes' the slowness by unpiping S3 and pushing chunks to it in a manual loop. What did they just break?
>
> They **defeated backpressure**. A manual `.write()` that ignores the **false** return value buffers unboundedly — the slow branch silently accumulates the whole file in RAM. The return-value contract *is* the flow control; bypassing it is how streaming OOMs come back.

> **Follow-up:** Two branches, very different speeds. One shared highWaterMark, or different ones?
>
> Independent buffers, tuned per branch. The slow S3 branch may want a **larger** highWaterMark to keep the pipe full across network jitter; the instant hash branch needs almost none. One global value either starves the network branch or wastes RAM on the fast one — match the buffer to each consumer's latency.

What sounds senior: Explaining that a fork runs at its **slowest consumer** and that backpressure propagates *back through the tee* to throttle the fast branch — then catching that a manual `.write()` defeats the flow-control contract — is the stream-internals depth most candidates only hand-wave.

DATA DELETION & THE RIGHT TO BE FORGOTTEN

Q16. A user invokes their right to be forgotten — purge everything tied to their content. Walk me through it across this pipeline.

Deletion is its own pipeline, not a single DELETE. It spans the dual store — the S3 object *and* the catalog row — so it carries the same orphan risk as the dual-write, handled the same way: a tombstone, a tracked compensating delete, the reconciler as backstop. The hard part is everything *downstream* of the primary stores — replicas, caches, search indexes, and the genuinely hard one, **backups**, which you cannot surgically edit.

> **Follow-up:** You cannot DELETE a row out of an immutable backup. So how is the user actually 'forgotten'?
>
> **Crypto-shredding.** Encrypt each user's content under a per-user key; deletion drops the key, so the backed-up ciphertext is unrecoverable without ever touching the backup. The honest alternative is a bounded **retention window** — backups age out in N days, so you commit to 'gone from live systems now, gone from backups within N days.' Both are defensible; claiming you surgically delete from a backup is not.

> **Follow-up:** Your reconciler deletes S3 keys with no DB row. Deletion removes the row first — does the reconciler fight you, or quietly do your job?
>
> That is exactly why deletion cannot lean on a side effect. I make it explicit: a **tombstone** the reconciler reads, so a deleted object is a coordinated state, not an accidental orphan it happens to clean up. A compliance guarantee resting on reconciler timing is how you fail an audit — the deletion path owns the delete end to end and emits a **proof-of-deletion** record.

What sounds senior: The level signal is treating deletion with the **same rigor as the write path** — dual-delete with compensation, an explicit tombstone the reconciler honors, an auditable proof record — plus an honest answer for backups (crypto-shred or retention) rather than hand-waving. Most candidates say 'delete the row' and miss that deletion is a pipeline, that backups are the hard part, and that a compliance promise cannot rest on a reconciler's timing.

MULTI-TENANT FAIRNESS

Q17. A shared pipeline serves many tenants, and one just queued ten million files in a single burst. Every other tenant's work is now stuck behind it. How do you stop one tenant from starving the others?

The root cause is a **single shared FIFO** — a FIFO is never fair under burst, so one tenant's flood owns the queue. The fix is **fair scheduling**, not a bigger queue: partition the work by a **tenant key** and have workers dequeue **round-robin (or weighted) across tenants** so no one backlog blocks another, plus a **per-tenant rate limit** — a token bucket per tenant — that throttles any single tenant to a fair share and buffers the excess instead of letting it block. Fairness is a *scheduling* property you build in, not something a FIFO hands you.

> **Follow-up:** A physical queue per tenant, for thousands of tenants?
>
> No — the isolation is *logical*. One queue **partitioned by tenant key** (Kafka partitions, or a tenant attribute on the message), and the **consumer** does the fair part: it dequeues round-robin across tenant keys and caps per-tenant in-flight work. The scheduling discipline lives in the consumer, not in thousands of physical queues — that's fairness that scales.

> **Follow-up:** And the burst that's already stuck right now?
>
> Triage first: lift the offending batch onto a **separate low-priority lane** so everyone else drains immediately, then let it catch up at a throttled rate. Isolate the noisy tenant now, fix the structure so it can't recur.

What sounds senior: The tell is naming fairness as a **scheduling discipline in the consumer** — partition by tenant key, round-robin, rate-limit each — rather than reaching for a bigger queue or a queue-per-customer. Juniors scale the queue; seniors schedule across tenants.

BLAST RADIUS WHEN DOWN

Q18. Your pipeline has been down for two hours and on-call is asking whether the device fleet is broken. What's the actual blast radius?

The senior move is separating 'the pipeline is down' from 'the system is broken' — they aren't the same, and saying why is the answer. Downstream is a **pull plus desired-state** model: devices run on their **last-reconciled state**, so an outage never touches what's already deployed — the fleet keeps running on its last-known-good config. Inbound work isn't lost either: uploads land in **durable storage** and the queue holds the events, so changes are **delayed, not dropped**, and drain when the pipeline recovers. So the real blast radius is bounded to one thing — *config changes stop propagating*. Nothing in flight is lost, nothing already live breaks. That's the line between an availability blip and an outage.

> **Follow-up:** What WOULD make it a real outage, then?
>
> A device needing a **fresh** config it has never seen — a brand-new device onboarding, or an urgent **security rotation** that has to reach the fleet *now*. For those, 'delayed' is 'broken.' So I name the exception: steady-state is fine, but time-critical pushes are the part of the blast radius that actually hurts — and that's what the SLO should be written against, not the average case.

> **Follow-up:** How do you stop the queue overflowing during a two-hour outage?
>
> The queue is durable and sized for backlog, so two hours of inbound buffers fine — that's the whole point of decoupling. The real risk is **recovery**: the pipeline comes back to a thundering backlog, so I drain at a **controlled rate** (bounded worker concurrency, not all-at-once) to keep recovery from hammering the downstream stores. Absorb during the outage, drain deliberately after.

What sounds senior: The tell is refusing the premise — 'down' is not 'broken.' A senior bounds the blast radius (steady-state keeps running, changes delayed not lost), names the one exception that genuinely hurts (time-critical pushes), and treats recovery as its own problem. Juniors say 'it's down, everything's broken'; seniors scope the damage.

TESTING & CONFIDENCE UNDER FAILURE

Q19. How would you test this pipeline? Specifically — how do you gain confidence it's correct under failure and concurrency, not just the happy path?

The happy-path tests are the easy part — the bugs live in redelivery, partial failure, and concurrency, so that's where the testing has to aim. I'd layer it: **unit** on the pure logic (the extension→handler dispatch, the idempotency-key derivation, the oldId→newId FK remap); **integration** against real storage and a real DB or localstack for the full upload→record path; and a **contract test on the strategy map itself** — assert every registered extension has a handler and every handler is registered, because that map drifting is exactly how an unknown type starts silently skipping. But the part that earns confidence is testing the *hard* properties. Fire the **same event twice, then N times concurrently**, and assert exactly one record and one set of derived artifacts — that's the test most teams skip and the one at-least-once delivery will eventually break in production. Then **fault injection**: kill the worker mid-handler, after the hash but before the DB write, and assert the retry converges to a clean state with no orphaned object. And run the **reconciler concurrently with live writes** over a seeded-inconsistent state, asserting it converges without double-deleting. The honest part: you can't test 'exactly once' into existence — so I test the guarantees that are actually real, **idempotency and convergence**, and use fault injection plus invariant assertions for the rest.

> **Follow-up:** What's the one test most teams skip that catches the most bugs?
>
> The **concurrent-duplicate** test — deliver the same event many times in parallel and assert a single record and a single set of side effects. Sequential redelivery is easy; the *race* — two workers on the same key at once — is where the idempotency check has a check-then-act gap, and at-least-once delivery guarantees you hit it eventually. If I write only one hard test, it's that one.

> **Follow-up:** How do you keep the reconciler test from being flaky?
>
> Make it **deterministic**: inject the clock and the 'in-flight vs orphaned' boundary so the decision is decidable instead of a race against wall-clock. Seed exact states — object-without-row, row-without-object — and assert convergence. For the concurrent case I drive a controlled interleaving, or run it many times under randomized scheduling and assert **invariants** (never double-delete, never drop a live object) rather than one exact trace. Test the property, not the output.

What sounds senior: The tell is aiming the tests at failure and concurrency rather than the happy path — plus the maturity to say out loud that some properties (exactly-once) can't be unit-tested, so you verify the weaker-but-real ones, idempotency and convergence, with fault injection and invariant assertions. Juniors recite the test pyramid; seniors name the *specific* hard test — the concurrent duplicate — and how they make non-determinism testable.

SHIPPING A CHANGE TO A LIVE PIPELINE

Q20. You need to ship a change to this pipeline — say a new field in the import format, or a change to how a handler writes records — while it's processing live traffic. How do you roll it out without downtime or corrupting data?

The insight that drives everything: in a rolling deploy, **old and new workers run at the same time** against the same queue, so for a window both versions are processing the same events. That kills the naive 'just deploy v2' — v2 can't emit anything v1 can't handle, or v1 chokes on v2's output mid-rollout. So I treat it as **expand, migrate, contract**: first deploy a version that *tolerates both* shapes — the reader ignores unknown fields and defaults missing ones, so it's backward- and forward-compatible; only once every worker can read the new shape do I flip the **writer** to emit it; and dropping the old path is a *third* deploy, after nothing produces the old shape anymore. For risk I **canary** — route one worker or a small slice of traffic to the new version, watch the error rate and the *derived-artifact correctness*, then ramp. And the change stays **idempotent and replay-safe**, so a worker picking up a half-processed event across the deploy converges instead of double-writing. The honest part is the **rollback asymmetry**: rolling back code is instant, but if v2 already *wrote* data v1 can't read — or wrote it wrong — a code rollback doesn't undo that. So the data change is forward-compatible by design, and anything destructive sits behind a **flag I can flip without a deploy**.

> **Follow-up:** Why not just deploy v2 everywhere at once?
>
> Because the queue holds in-flight events and workers don't cut over atomically — even a fast deploy has a window where old and new coexist, and any consumer lag widens it. If v2 writes a record shape v1 doesn't understand, v1 errors on — or silently mishandles — every event it picks up in that window. Expand/contract removes the assumption that there's ever a clean instant where only one version runs.

> **Follow-up:** The canary looks clean, you ramp to 100%, and an hour later you find it was writing a subtly wrong field. Now what?
>
> Code rollback stops the bleeding but doesn't fix the hour of bad records — that's the asymmetry. So flip the writer back (or kill the flag), then run a **targeted backfill** over just the affected window, which I can scope because records are timestamped and the change is identifiable. That's exactly why the pipeline keeps enough provenance to ask 'everything written by v2 between T1 and T2,' and why an irreversible write worries me more than a reversible one.

What sounds senior: The tell is knowing a rolling deploy means old and new run *concurrently*, so changes must be backward- and forward-compatible — expand, migrate, contract, never big-bang — and being honest that **rolling back code is not rolling back data**. Juniors say 'canary then ramp'; seniors design the change so both versions coexist safely and so a bad write is *findable and reversible*, because the data is the part you can't just redeploy.

COMPRESSING THE EXPORT

Q21. That 1,000,000-row CSV is ~400 MB on the wire, and the terminals pulling it are on metered links. How do you cut the transfer?

Compress the stream, don't buffer-then-compress. Pipe the cursor through the CSV transform, then through `zlib.createGzip()`, then to the response with `Content-Encoding: gzip` — so the gzip runs *inside* the same backpressured pipeline and memory stays constant. CSV is highly compressible (repeated delimiters, low-cardinality columns), so 400 MB routinely drops to 30–50 MB. The cost is server CPU; the win is bandwidth, transfer time, and — for an S3-staged export — storage and egress. You compress *as you stream*, never gzip a 400 MB buffer you first built in RAM.

> **Follow-up:** Gzip is CPU. When is compressing actually the wrong call?
> When the client can't decompress, when the payload is small enough that the round-trip saving is noise, or when you're CPU-bound and bandwidth-rich — then the compression tax costs more than it saves. It's also wrong on *already-compressed* or high-entropy content (a column of random UUIDs, binary blobs) where gzip spends CPU for near-zero gain. The rule: compress high-volume, high-redundancy, bandwidth-constrained transfers; skip it when the data is incompressible or the link is already fast and the CPU isn't spare. A big low-cardinality CSV over metered mobile is squarely worth it.

> **Follow-up:** You stage the export to S3 first, then hand out a presigned URL. Do you gzip before or after the upload?
> **Before** — store the object already-gzipped (`export.csv.gz`), so you pay compression once at write time and every download is small. Set the object's `Content-Encoding: gzip` so a browser client transparently inflates it, or hand the `.gz` to a device that inflates explicitly. Compressing after — letting a CDN or the client compress on the fly — re-pays CPU per download and doesn't shrink what's stored. Compress once at the source of truth, serve the small object many times: the same reason you don't re-render a cached artifact on every read.

What sounds senior: Piping the cursor through `gzip` *inside* the backpressured stream (constant memory, `Content-Encoding: gzip`) rather than compressing a buffered blob — and knowing when compression is the wrong call (incompressible data, CPU-bound) and to compress-once at the S3 source — is the transfer-optimization depth a senior round rewards.

RESUMABLE EXPORT

Q22. The export streams for four minutes. At row 800,000 the terminal's connection drops. Does it re-download the whole file, or can it resume?

A plain streaming response **restarts from zero** — the cursor was tied to that connection, so a drop means re-run. To make it resumable you **decouple the export from the delivery**: stage the export to S3 as an object, then hand out a presigned URL, and S3 serves it with **HTTP Range support** — the client re-requests `Range: bytes=N-` and resumes from where it stopped. For a live (non-staged) stream you'd instead need a **resume token** — checkpoint the last-emitted primary key, and on reconnect continue *after* that key, which works precisely because the source read is an **ordered keyset**, not an OFFSET. The staged-object-plus-Range path is simpler and the one I'd reach for.

> **Follow-up:** Why is the presigned-S3 path usually better than a resume token on the live stream?
>
> Because it makes resumption **S3's problem, not yours**: Range requests, retries, and partial delivery are built in and battle-tested, and the export is computed *once* into a stable object instead of re-running the query on every reconnect. A live resume token means re-issuing `WHERE id > last_key` each time — correct only if the ordering is stable and nothing shifts the keyset under you — and it pins a DB connection for the whole slow download. Staging decouples 'produce the data' from 'deliver the bytes,' which is why it's the pattern for anything big and slow. You trade a little storage and latency-to-first-byte for robust, connection-independent delivery.

> **Follow-up:** A resume token keyed on the last primary key — what breaks it, and how do you keep it correct?
>
> It breaks if the result set isn't **stably ordered**. Order by an immutable, unique key (the primary key), not a mutable column, so 'after key X' is well-defined and can't skip or repeat rows. Rows inserted *after* the checkpoint with a higher key show up on resume (usually fine — the export reflects a moving table); a truly point-in-time export instead needs a **snapshot** — a repeatable-read transaction or an as-of timestamp — rather than a naive resume. The token is only as consistent as the ordering and isolation behind it, which is exactly why staging a snapshot into S3 sidesteps the whole class of problems.

What sounds senior: Recognizing that a plain streamed export restarts from zero, and that resumption means **decoupling produce-from-deliver** — stage to S3 and let Range requests handle resume, or checkpoint an *ordered keyset* for a live token — plus the consistency caveat (stable ordering / snapshot isolation) is the delivery-robustness thinking that separates a senior answer from 'the user just retries.'
